The Journal of Open Source Software

orbvis: A Python CLI for Orbital-Projected Band Structure and DOS in DFT

Taradutt Pattnaik [1¶] **and S. Pamir Alpay** [1,2,3]

1 Department of Materials Science and Engineering, University of Connecticut, Storrs CT 06269 – USA **2** Department of Physics, University of Connecticut, Storrs CT 06269 – USA **3** Institute of Materials Science, University of Connecticut, Storrs CT 06269 – USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 28 October 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Density Functional Theory (DFT) is a widely used computational technique in materials science and condensed matter physics, allowing researchers to study electronic structure of atoms, molecules, and solids. DFT softwares generate large, detailed output files that contain extensive information about the energy levels and properties of electronic states of materials. However, interpreting these data-rich outputs files can be challenging, especially when additional features like orbital projection are enabled.

For example, in the *VASP* (Kresse & Furthmüller, 1996) software, setting `LORBIT = 11` in the input file (`INCAR`) instructs the program to output the projection of each electronic state onto the atomic orbitals of every atom in the system. While this orbital-resolved information provides detailed insights into how individual atomic orbitals could be contributing to the electronic structure, it also makes the output files significantly larger and more complex to parse and visualize without specialized tools.

orbvis, short for orbital visualization, is a Python based command-line tool that is designed to streamline this process. It extracts, processes and visualizes orbital projected band structures and projected density of states (pDOS) for individual atoms or groups of atoms, directly from DFT output files. Currently, *orbvis* supports only VASP and operates in two modes:

- **band** (requires `PROCAR` and plots orbital-projected band structures)
- **dos** (requires `DOSCAR` and plots projected density of states)

Users can run *orbvis* entirely from the terminal without requiring any prior knowledge of python. *orbvis* uses a configuration file inspired by *VASP* input file (`INCAR`) making it intuitive for researchers already familiar with the *VASP* environment. *orbvis* generates high-quality, publication-ready plots with detailed customization options for orbital selection, color schemes, and layout.

Statement of need

While several tools exist for post-processing DFT output files such as *p4vasp*(Dubey, 2025), *sumo*(Ganose et al., 2018), *pyprocar*(Herath et al., 2020), orbital-projected band structure and projected density of states (pDOS) visualization are often features among many. Each tool offers valuable features but often involves trade-offs in terms of flexibility, ease of use, or the clarity of the visualizations produced.

For example, *p4vasp* provides a graphical interface suitable for interactive plotting but lacks support for scripting or command-line workflows. *Sumo*, on the other hand, is a command-line

tool that integrates tightly with the *pymatgen*(Ong et al., 2013) and *seekpath*(Hinuma et al., 2017) ecosystems. However, its reliance on internally generated k-point paths makes it less adaptable to externally generated paths such as those from *VASPKIT*(Wang et al., 2021) or other k-path generators. In certain cases, like hybrid functional (HSE(Heyd et al., 2003)) calculations where the band structure and self-consistent field (SCF) k-points may be mixed in the same file, only the zero-weighted k-points are relevant for the band path. Tools that assume uniform formatting may fail or require manual intervention. Additionally, when band paths contain discontinuities, correct trimming and merging is needed to avoid artificial jumps, something many tools do not handle automatically.

orbvis is designed specifically to address these pain points. It is a lightweight, memory-efficient, and scriptable command-line tool focused solely on orbital-projected band structure and pDOS visualization from *VASP* output files. Unlike other tools, *orbvis* performs a series of automated preprocessing steps to ensure clean band structure visualization. It collects the k-point path, removes duplicates, filters out irrelevant points (e.g., weighted k-points in hybrid functional (HSE) calculations where band structure and SCF k-points can be mixed), and automatically detects and labels high symmetry points. Discontinuities along the k-path are identified and trimmed. The orbital-projected band data is then cut at these points and merged in an elegant way to produce a neat band structure plot.

orbvis improves the clarity of orbital-projection plots through a **scatter-based visualization** approach:

- The size of each scatter point is scaled according to the orbital contribution values from the `PROCAR` file.
- Adjustable transparency ensures that smaller contributions remain visible, even when overlaid by dominant orbitals.

This strategy enables simultaneous, uncluttered visualization of multiple orbitals and atoms a scenario where color interpolation schemes used by other tools often fall short.

Key Features of `orbvis`:

- **Command-line driven**: Designed for scripting, automation, and easy integration into computational workflows.
- **VASP-style configuration file**: A familiar `KEY = VALUE` format (modeled after *VASP* `INCAR`) allows users to set up complex projection and plotting tasks without any Python knowledge.
- **Robust k-point handling**:
  - Filters only those k-points that are required for band structure
  - Automatically detects and merges discontinuous segments in complex k-paths to eliminate visual jumps.
- **Spin support**: Handles spin-polarized and spin–orbit-coupling (SOC) enabled calculations via the `ISPIN` and `SOC` tags.
- **Memory-efficient implementation**: Uses *NumPy* (Harris et al., 2020) vectorization and optimized parsing to process large files with minimal memory usage.
- **Dot-based orbital projection plots**: Provides clear, uncluttered visualization of orbital contributions across atoms and orbitals.
- **Flexible color customization**:
  - Auto-generates color schemes using *Distinctipy* (Jack et al., 2024) and *NumPy* palettes such as `tab10`, `tab20`, `set10`, and `plasma`.
  - Accepts user-defined color lists via hex codes or color names.
- **High-symmetry point labeling**: Supports Unicode high-symmetry point labels (e.g., Γ, Λ, Σ) directly from terminal input, with automatic fallback to labels like K0, K1, etc., if none are specified.
- **Visualization of DOS**: Offers similarly configurable and high-quality DOS visualizations.

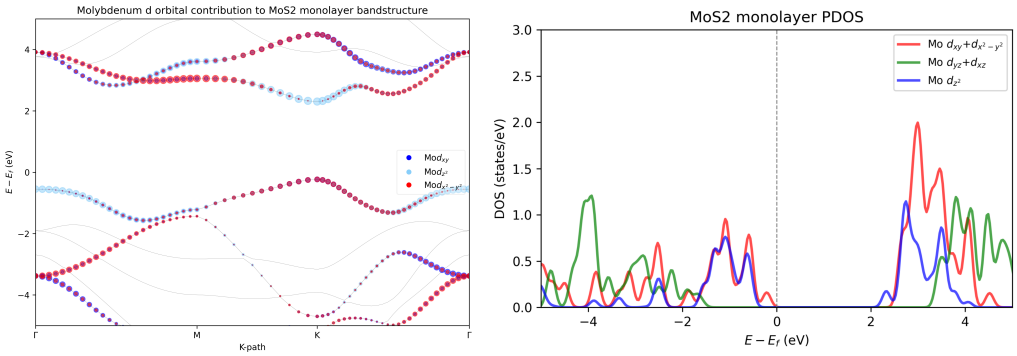

Figure 1: Plots generated with *orbvis*. **Left:** Band structure of monolayer MoS_2 showing contributions from Mo *d* orbitals, calculated with the HSE hybrid functional in *VASP*, reproducing results similar to (Chang et al., 2013). **Right:** Projected density of states (pDOS).

Rather than being a general-purpose toolkit, *orbvis* is designed to do one thing well: generate clean, interpretable, and customizable orbital-resolved band structure and DOS plots from raw DFT outputs. It is especially useful for systems where orbital characters reveal key electronic insights, such as 2D materials including transition metal dichalcogenides and their van der Waals heterostructures(Sun et al., 2017), spin–orbit coupling materials like topological insulators(Acosta et al., 2018; M. & John, 2022; Reid et al., 2020), magnetic systems(Devaraj et al., 2024; Jiang et al., 2018), and unconventional superconductors(Liu et al., 2018). By combining clarity, automation, and scripting compatibility, *orbvis* fills a niche in the DFT post-processing ecosystem, ideal for both exploratory studies and high-throughput workflows.

Acknowledgements

The authors acknowledge support from the high-performance computing center at the University of Connecticut for providing the computational resources.

References

Acosta, C. M., Lima, M. P., Silva, A. J. R. da, Fazzio, A., & Lewenkopf, C. H. (2018). Tight-binding model for the band dispersion in rhombohedral topological insulators over the whole brillouin zone. *Physical Review B*, *98*, 035106. https://doi.org/10.1103/PhysRevB.98.035106

Chang, C.-H., Fan, X., Lin, S.-H., & Kuo, J.-L. (2013). Orbital analysis of electronic structure and phonon dispersion in MoS2, MoSe2, WS2, and WSe2 monolayers under strain. *Physical Review B*, *88*, 195420. https://doi.org/10.1103/PhysRevB.88.195420

Devaraj, N., Bose, A., & Narayan, A. (2024). Interplay of altermagnetism and pressure in hexagonal and orthorhombic MnTe. *Physical Review Materials*, *8*, 104407. https://doi.org/10.1103/PhysRevMaterials.8.104407

Dubey, O. (2025). *p4vasp*.

Ganose, A. M., Jackson, A. J., & Scanlon, D. O. (2018). Sumo: Command-line tools for plotting and analysis of periodic ab initio calculations. *Journal of Open Source Software*, *3*, 717. https://doi.org/10.21105/joss.00717

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T.

DRAFT

E. (2020). Array programming with NumPy. *Nature*, *585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

Herath, U., Tavadze, P., He, X., Bousquet, E., Singh, S., Muñoz, F., & Romero, A. H. (2020). PyProcar: A python library for electronic structure pre/post-processing. *Computer Physics Communications*, *251*, 107080. https://doi.org/10.1016/j.cpc.2019.107080

Heyd, J., Scuseria, G. E., & Ernzerhof, M. (2003). Hybrid functionals based on a screened coulomb potential. *The Journal of Chemical Physics*, *118*, 8207–8215. https://doi.org/10.1063/1.1564060

Hinuma, Y., Pizzi, G., Kumagai, Y., Oba, F., & Tanaka, I. (2017). Band structure diagram paths based on crystallography. *Computational Materials Science*, *128*, 140–184. https://doi.org/10.1016/j.commatsci.2016.10.015

Jack, R., Jon, C., Kian-Meng, A., & Yannick, B. (2024). *Alan-turing-institute/distinctipy: v1.3.4*. Zenodo.

Jiang, Z., Wang, P., Jiang, X., & Zhao, J. (2018). MBene (MnB): A new type of 2D metallic ferromagnet with high curie temperature. *Nanoscale Horizons*, *3*, 335–341. https://doi.org/10.1039/C7NH00197E

Kresse, G., & Furthmüller, J. (1996). Efficient iterative schemes for ab initio total-energy calculations using a plane-wave basis set. *Physical Review B*, *54*, 11169–11186. https://doi.org/10.1103/PhysRevB.54.11169

Liu, D., Li, C., Huang, J., Lei, B., Wang, L., Wu, X., Shen, B., Gao, Q., Zhang, Y., Liu, X., Hu, Y., Xu, Y., Liang, A., Liu, J., Ai, P., Zhao, L., He, S., Yu, L., Liu, G., … Zhou, X. J. (2018). Orbital origin of extremely anisotropic superconducting gap in nematic phase of FeSe superconductor. *Physical Review X*, *8*, 031033. https://doi.org/10.1103/PhysRevX.8.031033

M., U., & John, R. (2022). Band inversion in Bi2Se3 from fat band analysis. *Journal of Crystal Growth*, *591*, 126719. https://doi.org/10.1016/j.jcrysgro.2022.126719

Ong, S. P., Richards, W. D., Jain, A., Hautier, G., Kocher, M., Cholia, S., Gunter, D., Chevrier, V. L., Persson, K. A., & Ceder, G. (2013). Python materials genomics (pymatgen): A robust, open-source python library for materials analysis. *Computational Materials Science*, *68*, 314–319. https://doi.org/10.1016/j.commatsci.2012.10.028

Reid, T. K., Alpay, S. P., Balatsky, A. V., & Nayak, S. K. (2020). First-principles modeling of binary layered topological insulators: Structural optimization and exchange-correlation functionals. *Physical Review B*, *101*, 085140. https://doi.org/10.1103/PhysRevB.101.085140

Sun, H., Wang, Z., & Wang, Y. (2017). Band alignment of two-dimensional metal monochalcogenides MXs (m=ga,in; x=s,se,te). *AIP Advances*, *7*. https://doi.org/10.1063/1.4986385

Wang, V., Xu, N., Liu, J.-C., Tang, G., & Geng, W.-T. (2021). VASPKIT: A user-friendly interface facilitating high-throughput computing and analysis using VASP code. *Computer Physics Communications*, *267*, 108033. https://doi.org/10.1016/j.cpc.2021.108033

DRAFT